



INSA ROUEN NORMANDIE

Département Informatique

PROJET INTÉGRATIF SMART ROBOT

Robot Autonome

Équipe projet :

Delacroix-Henrion Mathieu

Diallo Alioune

Malherbe Ylann

Mathieu Ilan

Olivier Neil

Schetrit Jahnis

ITI3 — Année 2025-2026

Décembre 2025

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Problématique	2
2	Partie électronique	2
2.1	Cahier des charges	2
2.2	Choix des composants	2
2.2.1	Capteur suiveur de ligne	3
2.2.2	Capteur RGB	3
2.2.3	Moteur Shield	3
2.2.4	Ecran LCD	4
2.2.5	Buzzer	4
2.3	Conception du Robot	5
2.3.1	Schéma électrique	5
2.3.2	Choix techniques	6
2.3.3	Estimation du robot	7
2.4	Difficultés rencontrées et pistes d'amélioration	8
3	Partie informatique	8
3.1	Cycle en V	8
3.1.1	Analyse	8
3.1.2	Conception préliminaire	9
3.1.3	Conception détaillée	11
3.1.4	Code C	18
3.2	Conclusion	18
3.2.1	Répartition du travail	18
3.2.2	Conclusion du projet	19
3.3	Bibliographie	19

1 Introduction

1.1 Contexte

Dans le cadre de notre premier semestre dans le département ISIA, nous avons pour but de réaliser un *Smart Robot* capable de se déplacer dans une ville modélisée sous forme de cases. Equipés de capteurs, moteurs et d'une raspberry Pi 4, notre objectif est de construire et coder un robot capable de parcourir un itinéraire imposés tout en déterminant le plus court chemin permettant de passer par un certain nombre de points obligatoires. C'est en combinant électronique, algorithmique et développement en C, ainsi que l'utilisation de GitLab que nous allons répondre aux attentes de ce projet. Ce rapport est structuré de la manière suivante : Dans un premier temps, nous présenterons la partie électronique contenant la description de la conception du robot avec le schéma électrique, les choix techniques que nous avons effectués, ainsi que les branchements et problèmes que nous avons pu rencontrés dans cette partie.

Dans un deuxième temps, nous présenterons la partie informatique, depuis l'analyse descendante du problème, jusqu'à l'implémentation des fonctions et procédures en C. Enfin, nous concluerons sur ce projet avec la répartition des tâches et ce que l'on a pu tirer de ce projet.

1.2 Problématique

La problématique principale de ce projet est de faire en sorte que le robot parcoure la ville de manière autonome et optimale en interprétant correctement les descriptions de la ville. En effet, le robot doit lire et traduire ces informations en une suite d'ordre (AV, TG, TD) lui permettant de se déplacer dans la ville. Celui-ci doit non seulement suivre les lignes au sol en faisant attention à corriger d'éventuelles déviations, mais aussi détecter et signaler les pastilles colorées signalant les points de passages obligatoires. En définitive, notre robot doit être capable de passer par les trois points obligatoires repérables par des pastilles, afficher sur l'écran la couleur de celle-ci tout en buzzant, afficher chaque intersection qu'il croise, ainsi que chaque action qu'il réalise tout le long du parcours (Avancer, Tourner Gauche, Tourner Droite).

2 Partie électronique

2.1 Cahier des charges

Pour la partie électronique, le cahier des charges est assez clair et précis car il était renseigné dans le sujet. La partie électronique se concentre sur le robot, ses composants et son fonctionnement. Les attendus sont les suivants : un robot qui suit un chemin tracé au sol, passant par des points précis, et qui revient à son point de départ. Lorsque le robot passera sur le point distinguable par une pastille de couleur, il émettra un son distinct et affichera la couleur de la pastille sur le LCD. Chacune de ses actions doivent être affichée en temps réel sur l'écran du LCD (Avancer, Virage à Gauche et Virage à Droite). L'arrivée qui est également le point de départ sera visible et détectable par le robot car il sera représenté par une pastille rouge.

2.2 Choix des composants

En ce qui concerne les moteurs, ceux-ci sont alimentés à l'aide de piles connectées en parallèles et sont reliés à un interrupteur. La raspberry Pi est alimentée par une batterie externe afin de garantir une autonomie et une alimentation stable. Le buzzer et les capteurs (suiveurs de ligne et RGB) sont connectés aux GPIO de la RaspberryPi et alimentés avec 3,3V. Concernant l'écran LCD, celui-ci nécessite une alimentation de 5V comme précisé dans sa documentation technique. Avant de monter intégralement les composants sur le robot, nous avons testés chacun des composants séparément : Une unité de test pour les moteurs (Avancer, reculer, s'arrêter...), une unité de test pour les capteurs (détecter une surface sombre, détecter une surface claire,

détecter une pastille de couleur...), ainsi qu'une unité de test pour l'affichage sur l'écran LCD (afficher 'Bonjour', effacer l'écran, afficher la couleur détectée par le capteur RGB...). Ces tests nous ont permis de d'ajuster l'emplacement des composants, la calibration des capteurs, de la même manière que trouver plus facilement ce qui pouvait ne pas fonctionner.

2.2.1 Capteur suiveur de ligne

Rôle

Les capteurs suiveurs de ligne permettent au robot de détecter et donc de suivre une ligne noire tracée sur une surface plus claire. L'objectif est de guider les mouvements du robot, de telle sorte que celui-ci corrige sa direction s'il n'est pas bien aligné sur la trajectoire de la ligne noire. En l'incluant dans notre projet, ce type de capteur nous permet de garder le robot sur la ligne noire, de détecter lorsque celui-ci dévie de la trajectoire, mais cela permet aussi de repérer les intersections et virages lui permettant de s'orienter.

Fonctionnement

Ces types de capteurs fonctionnent de la manière suivante :

Le noir absorbe la lumière tandis que le blanc ou les couleurs plus claires la réfléchissent. Les capteurs de ligne sont alors composés d'une LED infrarouge qui éclaire le sol en continu et d'une photodiode mesurant la lumière renvoyée. Concrètement, si un capteur suiveur de ligne se trouve sur une surface claire, alors la lumière infrarouge est fortement réfléchi. Le capteur reçoit donc beaucoup de lumière et envoie le signal indiquant qu'il s'agit d'une surface claire. Au contraire, sur une surface noire, la lumière est absorbée donc le capteur ne reçoit que très peu de lumière. Celui-ci envoie alors le signal indiquant qu'il s'agit d'une surface noire.

2.2.2 Capteur RGB

Rôle

Le capteur de couleur TCS 34725 permet au robot de détecter la couleur des pastilles présentes sur le parcours comme la pastille verte indiquant le point de départ du robot. Ce capteur est très important car il permet d'identifier correctement les points de passage, mais aussi d'afficher la couleur que ce capteur détecte sur l'écran LCD. Il est essentiel d'inclure ce capteur, sans quoi, notre robot ne serait pas capable de repérer les points de passages obligatoires du parcours.

Fonctionnement

Le capteur de RGB, comme son nom l'indique, permet de repérer trois couleurs : rouge, vert et bleu. Le TCS 34725 possède une petite LED blanche pour éclairer la surface et alors mesurer l'intensité lumineuse pour chacune des composantes (composantes rouge, ...). Ensuite, les valeurs sont converties en valeurs numériques via le moyen de communication I2C qui sont ensuite lus par la carte embarquée de la Raspberry Pi 4. Les trois intensités sont alors combinées pour déterminer de quelle couleur il s'agit.

2.2.3 Moteur Shield

Rôle

Le moteur driver est un intermédiaire entre la Raspberry Pi 4 et les moteurs du robot car il permet de fournir assez de courant aux moteurs, régler la vitesse, contrôler le sens de rotation des roues mais aussi de protéger la Raspberry car celle-ci ne peut pas alimenter directement les moteurs. En effet, la Raspberry ne fournirait que 3.3V ou au maximum 5V pour les moteurs tandis que ceux-ci peuvent être alimentés jusqu'à 12V. Il est donc indispensable d'utiliser un moteur shield.

Fonctionnement

Un moteur shield contient 2 canaux moteurs (moteur 1, moteur 2) ainsi que 6 broches de contrôle :

- Enable1 : permet de régler la vitesse du moteur 1

- Enable2 : permet de régler la vitesse du moteur 2
- IN1, IN2 : permettent de contrôler le sens du moteur 1
- IN2, IN3 : permettent de contrôler le sens du moteur 2

Afin de contrôler correctement du moteur, il est important de savoir comment fonctionnent ces différentes broches. Tout d'abord, pour contrôler la vitesse d'un moteur, nous avons besoin du PWM (Pulse Width Modulation) qui permet de simuler une tension variable à partir d'un signal (HIGH ou LOW). Lorsque l'on met cette valeur à HIGH(1), le moteur est activé à pleine vitesse. Inversement, lorsque l'on met cette valeur à LOW(0), le moteur est désactivé. Enfin, en ce qui concerne les broches permettant de contrôler le sens du moteur, celles-ci fonctionnent comme suit :

IN1	IN2	Action
0	0	rotation dans le sens 1
0	1	rotation dans le sens 2
1	0	arrêt
1	1	frein

TABLE 1 – Fonctionnement des broches

2.2.4 Ecran LCD

Rôle

Un écran LCD I2C permet d'afficher les informations provenant du robot comme les indications concernant son trajet sur le parcours (Tourner gauche, intersection. . .) Mais aussi d'afficher la couleur des pastilles située sur les points de passage obligatoires. Concrètement, il s'agit de l'interface entre le robot et nous, utilisateur.

Fonctionnement

Un écran LCD utilise beaucoup de fils, pour gérer cela, nous utilisons un LCD avec l'I2C pour réduire à seulement 2 fils de communication. Le câblage est alors plus simple et cette configuration est idéale pour notre robot surtout quand on utilise déjà beaucoup de GPIO avec les capteurs et moteurs. Seulement 4 lignes sont utilisées :

- SDA : données
- SCL : horloge
- VCC : alimentation
- GND : masse

Il est intéressant de noter que tous les périphériques I2C sont connectés en parallèle sur SDA/SCL. C'est exactement ce que l'on a pu faire avec le capteur RGB. L'écran LCD fonctionne de la manière suivante : Le micro contrôleur envoie les commandes comme le texte à afficher ou encore effacer l'écran. Le module I2C lui, reçoit les données et les convertit en signaux compréhensibles par le LCD.

2.2.5 Buzzer

Rôle

Le Buzzer a servi d'émettre un son lorsque le robot passe sur une pastille. Concrètement c'est un moyen pour le robot de nous confirmer qu'il a bien vu la pastille.

Fonctionnement

Le buzzer est un composant simple qui ne demande pas de protocole de communication compliqué. Pour le gérer, nous utilisons directement un GPIO. Le câblage est rapide et cette configuration est pratique. Seulement 3 lignes sont utilisées sur le module :

- S : signal de commande
- VCC : L'alimentation
- GND : La masse

2.3 Conception du Robot

2.3.1 Schéma électrique

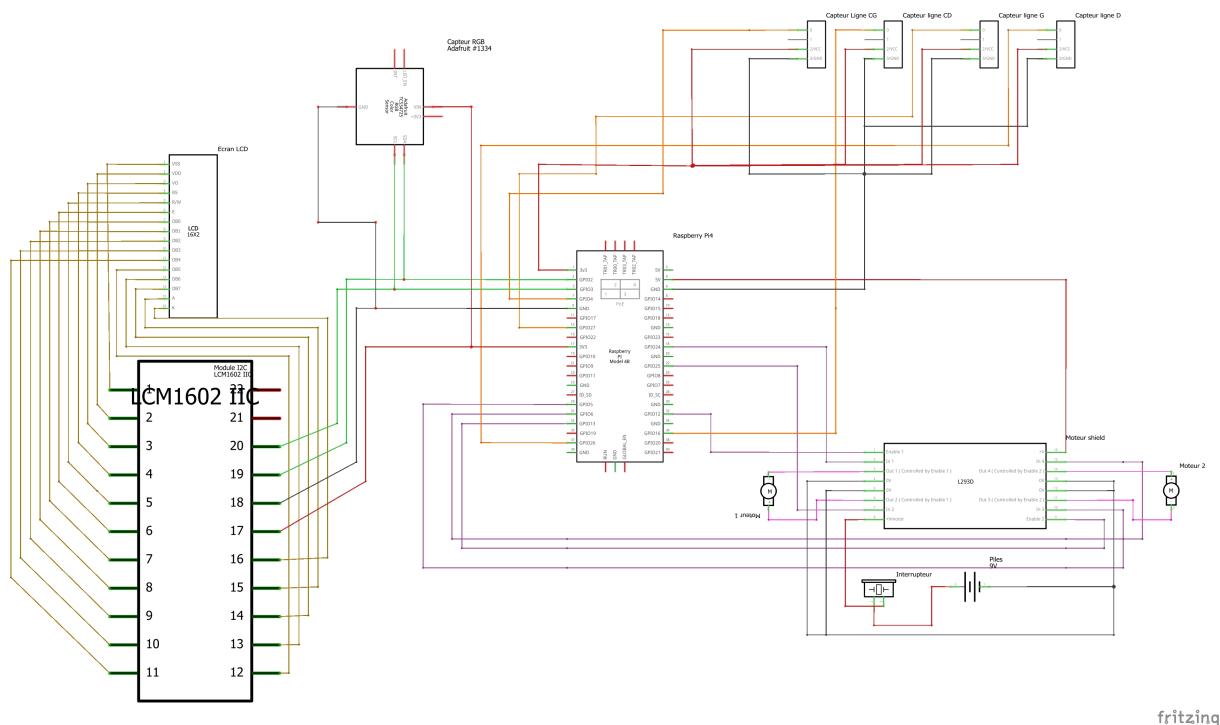
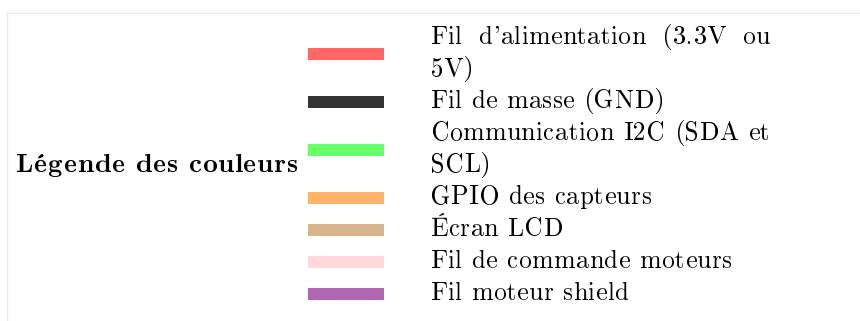


FIGURE 1 – Schéma électrique du robot autonome



2.3.2 Choix techniques

Pour le montage du robot, nous avons commencé par nous rassembler et discuter des différents composants que nous pouvions inclure au robot. Tout d'abord, nous avons discuté du meilleur emplacement pour les capteurs, essentiels au projet pour détecter les lignes noires du parcours. Après plusieurs propositions, nous avons finalement décidé de positionner deux capteurs proches l'un de l'autre en avant pour pouvoir suivre correctement la ligne noire et alors détecter par la même occasion lorsque le robot dévie. De plus, nous avons choisi de placer deux capteurs sur les cotés, un capteur sur le côté droit, ainsi qu'un capteur sur le côté gauche afin de détecter les intersections et virages.

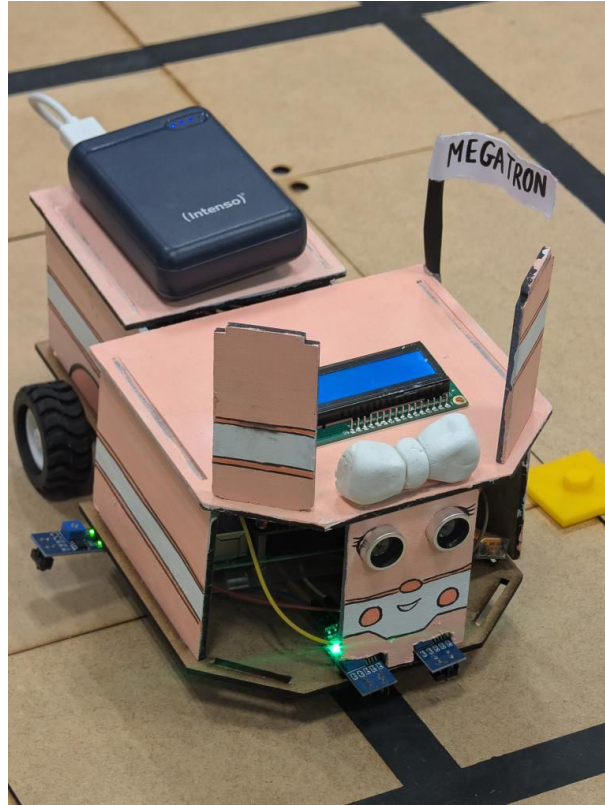


FIGURE 2 – smart robot personnalisé

2.3.3 Estimation du robot

Pour estimer les différentes valeurs de puissances consommées par notre robot nous sommes allée chercher les valeurs dans les datasheets associés au composants sur leurs différents site de vente ou sur le site du producteur, si un datasheet de composant n'est pas disponible ou trouvable alors nous prenons alors une valeurs moyennes des produits similaires que nous trouvons.

Appareil	Tension (V)	Courant (mA)	Puissance (W)
TCS34725	3,3	0,33	$1,09 \times 10^{-3}$
Capteur de ligne x4	3,3	20	0,264
L293D x2	3,3	600	3,96
LCD1602	5	21	0,105
Buzzer actif	3,3	30	0,099
PCF8574	3,3	0,04	$0,132 \times 10^{-3}$
Raspberry Pi	5	1 200	6

TABLE 2 – Bilan de consommation du robot

Ainsi pour estimer l'autonomie de notre batterie nous allons simplement venir diviser la capacité de la batterie par le courant total consommé par les différents composants connectés à la batterie.

Les composants lié a la batterie sont le buzzer actif, les 4 capteurs de lignes et le capteur RGB, les 2 contrôleur de pilotes de moteur, ainsi que le l'écran LCD qui sont connecté au GPIO de la Raspberry qui est elle même connecté à la batterie. Ainsi la consommations estimé des

composants est de environ 10,4 W, dont 4,32 W pour les éléments alimentés en 3,3 V et 6,11 W pour ceux alimentés en 5 V.

nb : Les moteurs étant alimentés séparément par des piles, leur consommation n'est pas incluse dans ce calcul.

L'autonomie du système peut donc être estimée à partir de l'énergie fournie par la batterie. Elle est donnée par le rapport entre l'énergie stockée dans la batterie et la puissance consommée par le système (environ 10,4 W hors moteurs). Ainsi, pour une batterie d'énergie 37.0 (en Wh), l'autonomie est approximativement $T=37.0/10,4=3h33min$. En tenant compte des pertes liées aux convertisseurs de tension, l'autonomie réelle est comprise entre 80 % et 90 % de cette valeur théorique.

2.4 Difficultés rencontrées et pistes d'amélioration

Nous avons rencontré de nombreuses difficultés, aussi bien sur le plan électronique que sur le plan organisationnel.

Tout d'abord, pour ce qui était de l'électronique, les premiers problèmes furent de s'accoutumer avec les composants. Pour la majorité, on ne les avait jamais vu avant et comprendre leur fonctionnement nécessite d'avoir un code lié et de pouvoir faire des tests, or nous n'avions pas de code pré-disposé. La suite fut de passer de la théorie à la pratique. En effet, lorsque nous avons codé le comportement du robot, nous nous sommes rendu compte de la différence entre notre comportement théorique et le fonctionnement réel. Une erreur dans la récupération de l'information des capteurs nous empêchait d'avancer. Un autre problème qui nous a beaucoup occupé fut la détection de couleur. Lors de nos tests préliminaire le capteur fonctionnait comme souhaité, mais en pratique non, et notre manière de trouver les bonnes valeurs pour le faire fonctionner reste assez hasardeuse. De plus, faire un montage stable pour le robot fut une réel épreuve, certains câbles ne se branchaient pas correctement et/ou se détachaient. Faire tenir les composants sur la structure du robot n'était pas de tout repos non plus. Sa structure aillant les dimensions minimales afin de tout accueillir, il est difficile de le monter rapidement et on a un manque de solidité.

De l'autre coté nous avons aussi eu des difficultés d'organisations, au début nous avons séparé le groupe en deux, puis très vite nous avons remarqué que nous perdions du temps et de l'efficacité, alors nous avons choisis de faire trois groupe de deux. Cela nous a permis de prêter plus de temps à la partie algorithmique tout en restant efficace sur la partie électronique.

Notre robot reste largement améliorable, dans un premier temps dans notre gestion des tâches et de l'effectif, mais surtout dans notre relecture du code et notre vigilance sur ce que l'on écrit. La détection de couleur pourrait être plus stable et efficace. Après de nombreux essais, le robot échoue parfois à accomplir sa tâche, cela est du à plusieurs paramètres comme l'éclairage de la pièce, ou encore la couleur du circuit.

3 Partie informatique

3.1 Cycle en V

3.1.1 Analyse

L'analyse a un rôle essentiel dans ce projet afin d'identifier les principaux problèmes que cause ce projet, et alors de traiter ces problèmes en sous problèmes plus simples à resoudre. Les principaux problèmes à resoudres sont :

- étudier le fichier `.txt` afin de récupérer les informations essentielles liées au robot et à la construction de la ville

- créer un graphe modélisant les liaisons entre les différents points de la ville
- calculer le chemin le plus court
- générer les ordres que le robot doit suivre sur la sortie standard

1. Types

Type Direction Ce type permet d'associer une direction(Sud, Nord, Ouest, Est) au robot et permet donc de faciliter l'interdiction des demi-tours quant à la direction actuelle du robot.

Type Ordre Ce type est une énumération qui permet de donner au robot les actions qu'il peut effectuer tout au long du parcours (Avancer, Tourner Droite, Tourner Gauche, Ne Rien Faire).

Type Robot Ce type sera défini sous forme d'une structure permettant de contenir la case actuelle ou se trouve le robot, ainsi que sa direction.

Type Chemin Ce type permettra de contenir l'ensemble des points par lequel le robot doit passer pour réaliser le chemin le plus court.

Type EtatDuDijkstra Ce type sera représenté par une structure permettant de renseigner le point actuel ou l'on se trouve, le point depuis lequel on arrive (point précédent) ainsi que le chemin pour parvenir jusqu'au point actuel.

Type Solution Ce type sera utilisé lors de la résolution du chemin le plus court, regroupant alors l'ordre dans lequel nous allons passer par les points obligatoires ainsi que le chemin complet des points par lesquels nous allons passer.

Type File Le dernier type que nous avons décidé d'implémenter est une structure File qui stockera les différents états du Dijkstra que l'on pourra obtenir grâce aux indices de début et fin de la file.

Pour ce qui est du Graphe, nous utilisons les TAD déjà implémentés de Monsieur Delestre utilisant de la même manière les TAD ListeChaineListe ainsi que tableHachage.

3. Analyse descendante

Une fois avoir pré-défini les types et structure que nous souhaitons mettre en place, nous avons étudié les algorithmes de parcours afin de choisir le plus adapté. Souhaitant représenter la ville sous forme de graphe, nous avons le choix entre deux algorithmes bien précis :

- **Dijkstra** : un algorithme efficace pour trouver le plus court chemin dans un graphe pondéré.
- **A*** : une version améliorée de Dijkstra, utilisant une heuristique pour optimiser la recherche.

Étant donné la petite taille du labyrinthe, nous avons opté pour un Dijkstra dont les distances entre sommets des graphes seraient tous égaux à un.

3.1.2 Conception préliminaire

1. TAD

TAD tDirection Nom : tDirection

Opérations :

- **directionOuest** : tDirection $\rightarrow$ tDirection
- **directionEst** : tDirection $\rightarrow$ tDirection

TAD Robot

Nom : Robot

Utilise :

- tDirection
- NaturelNonNul

Opérations :

- **setDirection** : tDirection $\times$ Robot $\rightarrow$ Robot
- **getDirection** : Robot $\rightarrow$ tDirection
- **getCaseRobot** : Robot $\rightarrow$ NaturelNonNul
- **setCaseRobot** : Robot $\times$ NaturelNonNul $\rightarrow$ Robot

TAD Chemin

Nom : Chemin

Utilise :

- NaturelNonNul

Opérations :

- **obtenirChemin** : Chemin $\rightarrow$ tableau[1.. NB_POINTS_MAX] de NaturelNonNul
- **obtenirNbPoints** : Chemin $\rightarrow$ NaturelNonNul
- **fixerNbPoints** : Chemin $\times$ NaturelNonNul $\rightarrow$ Chemin

TAD File

Nom : File

Utilise :

- EtatDuDijkstra
- Naturel

Opérations :

- **initialisation_file** : $\rightarrow$ File
- **estVideFile** : File $\rightarrow$ Naturel
- **estPleineFile** : File $\rightarrow$ Naturel
- **enfiler** : File $\times$ EtatDuDijkstra $\rightarrow$ File
- **defiler** : File $\rightarrow$ File $\times$ EtatDuDijkstra
- **liberer_file** : File $\rightarrow$

Préconditions : enfiler($f, pDij$) : non estPleineFile(f)

Préconditions : defiler(f) : non estVideFile(f)

2. Signatures des fonctions

Dans cette sous partie, nous établirons les signatures de toutes les fonctions et procédures utilisées dans la partie algorithme du projet. Toutefois, pour la partie conception détaillée, nous ne présenterons que les algorithmes non triviaux et essentiels à notre résolution du problème principal : Trouver le chemin le plus court.

Fonctions et procédures du TAD Robot

- Fonction directionOuest(direction: tDirection) : tDirection
- Fonction directionEst(direction: tDirection) : tDirection
- Fonction getDirection(r: Robot) : tDirection
- Procédure setDirection(E/S r: Robot, E direction: tDirection)
- Fonction getCaseRobot(r: Robot) : NaturelNonNul
- Fonction setCaseRobot(E/S r: Robot, E nouvelleCase: NaturelNonNul)

Fonctions et procédures du TAD Chemin

- Fonction obtenirNbPoints(c: Chemin) : naturel

- Procédure obtenirChemin(E c: Chemin, S tabPoints: tableau[1..NB_POINTS_MAX] de NaturelNonNul)
- Procédure fixerNbPoints(E/S c: Chemin, E nombre_points: NaturelNonNul)

Fonctions et procédures du TAD File

- Fonction initialisation_file(): File
- Fonction estVideFile(f: File) : naturel
- Fonction estPleineFile(f: File) : naturel
- Procédure enfiler(E/S f: File, E etat: EtatDuDijkstra)
- Fonction defiler(f: File): EtatDuDijkstra
- Procédure liberer_file(E f: File)

Fonctions et procédures du graphe et de Dijkstra

- Procédure recuperationInfoFichier(E fileNameVille: ChaîneDeCaractères, S l, nb_liaisons, nbCasesObligatoires, caseInitRobot: NaturelNonNul, tabCasesObligatoires: tableau[1..MAX_CASES_OBLIGATOIRES], tabPourGraphe[NB_POINTS_MAX][2], orientationInitRobot: tDirection): File
- Fonction creationGrapheVille(tabLiaisonsVille: tableau[1..NB_POINTS_MAX][2] de NaturelNonNul, nbLiaisons: NaturelNonNul) : G_Graphe
- Fonction dijkstra_avec_exclusions(g: G_Graphe, depart, arrivee, nb_a_eviter : NaturelNonNul, sommet_a_eviter : tableau[1..NB_POINTS_MAX] de NaturelNonNul) : Chemin
- Fonction dijkstra(g: G_Graphe, depart, arrivee: NaturelNonNul)
- Procédure permutation(E arr: tableau[3] de NaturelNonNul, S permutations[6][3])
- Fonction chemin_autorise(c: Chemin):
- Fonction verifier_jonction_segment(segment1, segment2: Chemin): Naturel
- Fonction resoudre_chemin_plus_court(g: G_Graphe, depart, longueur_ville: NaturelNonNul, directionInitialeRobot: tDirection, point_obligatoires: tableau[3] de NaturelNonNul)

Fonctions et procédures du fichier d'ordres

- Fonction ordreDuChangementDeDirection(direction, nlleDirection: tDirection)
- Procédure determinerOrdre(E/S robot: Robot, E caseSuivanteRobot, largeurCircuit: NaturelNonNul, S ordre1, ordre2: Ordre)
- Procédure determinerOrdres(E c: Chemin, grapheVille: G_Graphe, robot: Robot, largeurCircuit: NaturelNonNul, S tabOrdres: tableau[NB_POINTS_MAX] d'Ordre)
- Procédure creationFichierOrdres(E grapheVille: G_graphe, plusCourtChemin: Chemin, largeurCircuit: NaturelNonNul, directionInitRobot: tDirection)

3.1.3 Conception détaillée

1. Types

Type tDirection = {NORD, SUD, EST, OUEST }

Type Ordre = {AV, TG, TD, NO }

Type Robot = structure **Champs** :

- **caseRobot** : Naturel Non Nul
- **direction** : tDirection

Type Chemin = structure **Champs** :

- **points** tableau[1..NB_POINTS_MAX] de NaturelNonNul
- **direction** : tDirection

Type EtatDuDijkstra = structure **Champs** :

```

— point_actuel : NaturelNonNul
— point_precedent : NaturelNonNul
— chemin : Chemin
Type Solution = structure Champs :
— ordre : tableau[1..3] de NaturelNonNul
— chemin_complet : Chemin
Type file = structure Champs :
— elemnts : tableau[1..MAX_FILE] de EtatDuDijkstra
— debut : NaturelNonNul
— fin : NaturelNonNul

```

Algorithme .1 : resoudreCheminPlusCourt

```
1 Entrée  $g$  : Graphe ( $G\_Graphe$ ),  $point\_obligatoires$  : tableau[3] de  $NNN$ ,  $depart$  :  
    $NNN$ ,  $longueur\_ville$  :  $NNN$ ,  $directionInitialeRobot$  :  $tDirection$   
2 Sortie  $solution$  :  $Solution$   
3 Début  
4    $solution \leftarrow \{0\}$   
5    $solution.chemin\_complet.nb\_points \leftarrow 0$   
6    $meilleure\_distance \leftarrow 99999$   
7    $point\_cles[0] \leftarrow depart$   
8   pour  $i \leftarrow 1$  à 3 faire  
9      $point\_cles[i] \leftarrow point\_obligatoires[i - 1]$   
10  fin  
11   $indices\_classement \leftarrow [1, 2, 3]$   
12   $permutations \leftarrow permutation(indices\_classement)$   
13  pour  $p \leftarrow 0$  à 5 faire  
14     $parcours \leftarrow [0, permutations[p][0], permutations[p][1], permutations[p][2], 0]$   
15     $distance\_totale \leftarrow 0$   
16     $estValide \leftarrow 1$   
17     $segments \leftarrow$  tableau de 4 Chemin  
18    pour  $i \leftarrow 0$  à 3 faire  
19       $index\_debut \leftarrow parcours[i]$   
20       $index\_fin \leftarrow parcours[i + 1]$   
21       $sommet\_depart \leftarrow point\_cles[index\_debut]$   
22       $sommet\_arrivee \leftarrow point\_cles[index\_fin]$   
23      si  $i > 0$  alors  
24         $sommets\_a\_eviter \leftarrow$   
           $segments[i - 1].points[segments[i - 1].nb\_points - 2]$   
25      sinon  
26        si  $directionInitialeRobot = SUD$  alors  
27           $sommets\_a\_eviter \leftarrow depart - longueur\_ville$   
28        sinon  
29          fin  
30         $directionInitialeRobot = NORD$   
           $sommets\_a\_eviter \leftarrow depart + longueur\_ville$   
31        si  $directionInitialeRobot = EST$  alors  
32           $sommets\_a\_eviter \leftarrow depart - 1$   
33        sinon  
34          fin  
35         $sommets\_a\_eviter \leftarrow depart + 1$   
36      fin  
37       $segments[i] \leftarrow$   
         $dijkstra\_avec\_exclusions(g, sommet\_depart, sommet\_arrivee, sommets\_a\_eviter, 1)$   
  
38      si  $segments[i].nb\_points = 0$  alors  
39         $estValide \leftarrow 0$   
40        interrompre  
41      fin  
42      si  $\neg chemin\_autorise(segments[i])$  alors  
43         $estValide \leftarrow 0$   
44        interrompre  
45      fin  
46       $distance\_totale \leftarrow distance\_totale + (segments[i].nb\_points - 1)$   
47    fin  
48    si  $estValide$  alors  
49      pour  $i \leftarrow 0$  à 2 faire  
50        si  $\neg verifier\_direction(segments[i], segments[i + 1])$  alors
```

Algorithme .2 : dijkstra_avec_exclusion

Entrée : $g : G_Graphe^*$, $depart :$, $arrivee :$,
 $sommets_a_eviter : NNN^*$, $nb_a_eviter : tableau[1..$

$NB_POINTS_MAX]$ de NNN

Sortie : $resultat : Chemin$

1 Début

```
2    $resultat : Chemin;$ 
3    $resultat \leftarrow \{0\}$   $points\_visites : tableau[NB\_POINTS\_MAX][NB\_POINTS\_MAX]$ 
   de  $NNN$ 
4    $i : NNN;$ 
5    $j : NNN;$ 
6   pour  $i \leftarrow 1$  to  $NB\_POINTS\_MAX$  faire
7       pour  $j \leftarrow 1$  to  $NB\_POINTS\_MAX$  faire
8            $points\_visites[i][j] \leftarrow 0$  fin
9       fin
10       $file : File;$ 
11       $file \leftarrow allouer(sizeof(File))$ 
12       $initialisation\_file(file);$ 
13       $initial : EtatDuDijkstra;$ 
14       $initial.point\_actuel \leftarrow depart$ 
15       $initial.point\_precedent \leftarrow 0$ 
16       $initial.chemin.points[0] \leftarrow depart$ 
17       $initial.chemin.nb\_points \leftarrow 1$ 
18       $enfiler(file, initial);$ 
19       $points\_visites[depart][0] \leftarrow 1$ 
20      tant que la file n'est pas vide faire
21           $etat : EtatDuDijkstra;$ 
22           $etat \leftarrow defiler(file);$ 
23          si  $etat.point\_actuel = arrivee$  alors
24               $resultat.nb\_points \leftarrow etat.chemin.nb\_points;$ 
25              pour  $i \leftarrow 1$  to  $etat.chemin.nb\_points$  faire
26                   $resultat.points[i] \leftarrow etat.chemin.points[i];$ 
27              fin
28               $liberer\_file(file);$ 
29              Retourner  $resultat;$ 
30          fin
31           $voisins : LCL\_Liste;$ 
32           $voisins \leftarrow G\_obtenirSommetsAdjacents(g, etat.point\_actuel);$ 
33          pour  $i \leftarrow 1$  to  $LCL\_longueur(voisins)$  faire
34               $voisin : NNN;$ 
35               $voisin \leftarrow LCL\_element(voisins, i);$ 
36              si  $voisin = etat.point\_precedent$  alors
37                  ;
38              fin
39               $est\_a\_eviter : 0;$ 
40              ( $k : NNN;$ 
41              )
42              pour  $k \leftarrow 1$  to  $nb\_a\_eviter$  faire
43                  si  $voisin = sommet\_a\_eviter[k]$  alors
44                       $est\_a\_eviter \leftarrow 1;$ 
45                  fin
46              fin
47              si  $est\_a\_eviter$  alors
48                  fin
49                   $deja\_dans\_chemin \leftarrow 0;$ 
50                  si  $voisin <> arrivee$  alors
```

Algorithme .3 : determinerOrdres

Entrée : c : Chemin, $grapheVille$: Graphe , $robot$: Robot, $largeurCircuit$: NNN

Sortie : $tabOrdre$: Tableau[1..NB_POINTS_MAX] d'ordre

```
1 Déclaration  $j, nbCaseChemin$  : NNN,  $ordre1, ordre2$  : Ordre,
2    $sommetsAdjacents$  : LCL_Liste,
3    $plusCourtChemin$  : Tableau[1..NB_POINT_MAX] de NNN
4 Début
5   obtenirChemin( $c, plusCourtChemin$ );
6    $nbCaseChemin \leftarrow obtenirNbPoints(c)$ 
7    $j \leftarrow 0$ 
8   pour  $i \leftarrow 0$  to  $nbCaseChemin - 1$  faire
9      $sommetsAdjacents \leftarrow G_{obtenirSommetsAdjacents}(grapheVille,$ 
10       $getCaseRobot(robot))$ 
11      $determinerOrdre(robot, plusCourtChemin[i + 1], largeurCircuit, ordre1, ordre2);$ 
12     si  $ordre1 == NO$  alors
13       si  $j \geq 1$  et  $tabOrdres[j - 1] == AV$  et  $LCL\_longueur(sommetsAdjacents)$ 
14          $> 2$  alors
15            $tabOrdres[j] \leftarrow ordre2$ 
16            $j \leftarrow j + 1$ 
17         fin
18       sinon si  $j == 0$  alors
19          $tabOrdres[j] \leftarrow ordre2$ 
20          $j \leftarrow j + 1$ 
21       fin
22     sinon
23        $tabOrdres[j] \leftarrow ordre1$ 
24        $j \leftarrow j + 1$ 
25        $tabOrdres[j] \leftarrow ordre2$ 
26        $j \leftarrow j + 1$ 
27     fin
28    $tabOrdres[j] \leftarrow NO$ 
29 retourner  $tabOrdre$ ;
```

Algorithme .4 : determinerOrdre

Entrée : *robot* : *Robot*,
 caseSuivanteRobot : *Naturel*,
 largeurCircuit : *Naturel*

Sortie : *ordre1* : *Ordre*,
 ordre2 : *Ordre*

1 Déclaration

2 *caseInitRobot* : *Naturel*,
3 *dirRobot* : *Direction*,
4 *diff* : *Naturel*,
5 *nlleDirection* : *Direction*

6 Début

```
7   caseInitRobot  $\leftarrow$  getCaseRobot(robot);  
8   dirRobot  $\leftarrow$  getDirection(robot);  
9   diff  $\leftarrow$  caseSuivanteRobot – caseInitRobot;  
10  nlleDirection : Direction;  
11  si diff = 1 alors  
12    | nlleDirection  $\leftarrow$  EST;  
13  fin  
14  sinon si diff = –1 alors  
15    | nlleDirection  $\leftarrow$  OU EST;  
16  fin  
17  sinon si diff = largeurCircuit alors  
18    | nlleDirection  $\leftarrow$  SUD;  
19  fin  
20  sinon si diff = –largeurCircuit alors  
21    | nlleDirection  $\leftarrow$  NORD;  
22  fin  
23  sinon  
24    | nlleDirection  $\leftarrow$  dirRobot;  
25  fin  
26  ordre1  $\leftarrow$  ordreDuChangementDeDirection(dirRobot, nlleDirection);  
27  setDirection(robot, nlleDirection);  
28  ordre2  $\leftarrow$  AV;  
29  setCaseRobot(robot, caseSuivanteRobot);
```

3.1.4 Code C

3.2 Conclusion

3.2.1 Répartition du travail

Tâches	Jahnis	Mathieu	Alioune	Ylann	Ilan	Neil
Construction du robot	X	X	X	X	X	X
Fonctions moteur	X	X	X		X	
Fonctions capteurs		X				X
Fonctions LCD		X	X	X		
Tests unitaires électronique	X	X			X	
Débogage électronique		X			X	
Fonctions liées à la détermination du plus court chemin	X					
Fonctions liées à la création du graphe						X
Tests unitaires algorithme	X					X
Débogage algorithme	X					X
Mise en place du makefile et bashrc				X		X
Rédaction du rapport	X	X	X	X	X	X

TABLE 3 – Répartition des tâches

3.2.2 Conclusion du projet

En conclusion, ce projet nous a permis de mettre nos connaissances en pratique afin de développer un robot autonome capable se déplacer selon un parcours donné. Les choix des composants ainsi que le choix de l'algorithme pour trouver le plus court chemin nous a permis d'atteindre un résultat plutôt satisfaisant en terminant deuxième du classement lors de la soutenance. Pendant le projet, nous avons fait face à de nombreux défis et de nombreuses difficultés comme la gestion du capteur RGB pour la détection des pastilles des couleurs. Toutefois, nous avons su nous communiquer et nous entraider pour surmonter ces difficultés et améliorer du mieux que l'on pouvait notre robot. Un point très important que nous souhaitons évoquer : les différences entre la théorie et la réalité. Effectivement, nous avons dès le début défini le nombre de composant que nous souhaitions utiliser ainsi que la manière dont nous souhaitions réaliser les choses. Néanmoins, nous avons été obligés de faire de nombreux ajustements après plusieurs tests, tant dans la partie électrique du robot, que dans la partie résolution du parcours. Finalement, ce projet nous a appris à travailler sur plusieurs aspects différents d'un même projet en répartissant le travail selon les préférences et forces de chacun. L'utilisation de Git a d'ailleurs été indispensable pour coordonner ce projet efficacement. Au terme de cette expérience, nous avons pu nous préparer aux attentes des entreprises et ce, malgré le délai assez court. Elle nous a permis de découvrir des méthodes de travail efficace et d'appliquer concrètement nos compétences théoriques.



FIGURE 3 – Groupe 15 du PI-smart robot

3.3 Bibliographie

Références

- [1] Adafruit, *TCS34725 Color Sensor Datasheet*. <https://cdn-shop.adafruit.com/datasheets/TCS34725.pdf>
- [2] Powertech, *Capteur de suivi de ligne infrarouge TCRT5000*. <https://powertech-dz.org/product/capteur-de-suivi-de-ligne-infrarouge-tcrt5000-ir-1-4-5-8-canaux/>
- [3] Texas Instruments, *L293D Motor Driver Datasheet*. https://moodle.insa-rouen.fr/pluginfile.php/191118/mod_resource/content/3/l293d.pdf
- [4] Adafruit, *LCD1602 Datasheet*. <https://cdn-shop.adafruit.com/datasheets/TC1602A-01T.pdf>
- [5] Valeurs moyennes issues de plusieurs sources en ligne (absence de documentation officielle).
- [6] Philips, *PCF8574 I/O Expander Datasheet*. <https://www.alldatasheet.fr/datasheet-pdf/view/424340/PHILIPS/PCF8574T.html>
- [7] Raspberry Pi Foundation, *Raspberry Pi 4 Datasheet*. <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>